

Final Guide

The five exercises on your paper — INHN0006

Unit testing (write it, then test it) · REST · Deployment diagram
Black-box testing · Fill in the blanks

Every step, every keyword, every trap

Prepared for Furgan Siyahov

Contents

Click any line to jump straight to it.

Your paper · the five exercises	3	§3 · Deployment diagram	12
§1 · Unit testing — write the code, then test it	4	Build order — six steps, in this sequence	12
Step 1 · The enum	4	The notation, drawn	12
Step 2 · The model class	5	The distinction that is always tested	13
Step 3 · The calculation that branches on the enum	5	Worked example — read the sentence, draw the element	13
Step 4 · The action that returns a boolean AND changes state	6	§4 · Black-box testing	15
Step 5 · The tests — both assertions, one method	7	Part 1 · Equivalence classes	15
§2 · REST — save, find, remove	9	Part 2 · One representative test case per class	15
The complete resource	9	Part 3 · Boundary values — the three-point rule	16
The six decisions that decide your marks	10	§5 · Fill in the blanks	17
		Last page · the nine lines	20

Your paper · the five exercises

Built from what your friend reported. Treat the details as approximate and the **shapes** as reliable — every one of these has appeared on a previous ISE paper in a near-identical form.

#	EXERCISE	WHAT IT REALLY TESTS	GO TO
1	Unit testing — implement, then test	Write the enum + class + calculation <i>yourself</i> , then test it. <code>assertTrue</code> for "did it succeed", <code>assertEquals</code> for "what changed" — both in the same method .	§1
2	REST — save · find · remove	Three endpoints, the right HTTP verb on each, id in the path, validation before the service call.	§2
3	Deployment diagram	3-D nodes, artifacts nested inside, plain labelled lines between nodes. Interfaces go between components, never between nodes.	§3
4	Black-box testing	Equivalence classes split per bracket, one representative test each, three-point boundaries for every parameter combination.	§4
5	Fill in the blanks	Single keywords from JUnit, EasyMock, Git, Gradle, UML and coverage. Learn the exact spelling.	§5

Before anything else: `git add .` && `git commit -m "start"` && `git push`. Then again every ten minutes. Artemis grades the pushed commit test-by-test — half-finished code scores half the tests; uncommitted code scores nothing.

§1 · Unit testing — write the code, then test it

This is the one to over-prepare. You are not handed a finished class to test: you build the **enum**, the **model class** and the **calculation**, and only then the tests. If the model is wrong the tests are worthless, so build it in this exact order.

Step 1 · The enum

Two forms. Read the task: if each constant carries a **number** ("long-haul consumes twice as much"), use the second.

```
// FORM A — plain constants. Use when the task only names categories.
public enum FlightType {
    SHORT_HAUL, LONG_HAUL, CARGO
}

// FORM B — constants WITH a value. Use when each one has a factor, rate or limit.
public enum FlightType {
    SHORT_HAUL(0.5),          // ← the ( ) is a call to the constructor below
    LONG_HAUL(2.0),
    CARGO(3.5);               // ← SEMICOLON after the last constant. Required.

    private final double fuelFactor;

    FlightType(double fuelFactor) {    // constructor is implicitly private
        this.fuelFactor = fuelFactor;
    }

    public double getFuelFactor() { return fuelFactor; }
}
```

Enum API you may need

<code>FlightType.CARGO</code>	reference a constant
<code>FlightType.values()</code>	array of every constant, in declaration order — for looping
<code>FlightType.valueOf("CARGO")</code>	String → constant. Throws <code>IllegalArgumentException</code> if no match — case sensitive
<code>type.name()</code>	constant → its exact String name
<code>type.ordinal()</code>	0-based position. Avoid relying on it; renumbering breaks silently
<code>==</code>	the correct way to compare enums. <code>equals()</code> also works, but <code>==</code> is null-safe and idiomatic

Spell the constants exactly as the task writes them, in `UPPER_SNAKE_CASE`. The hidden tests reference them by name, so `LongHaul` instead of `LONG_HAUL` fails to compile against the test file.

Step 2 · The model class

```
public class Spaceship {

    private final String name;           // final: set once, never changes
    private final FlightType flightType;
    private double fuel;                 // NOT final: this one changes

    public Spaceship(String name, FlightType flightType, double fuel) {
        if (name == null || name.isBlank()) {           // validate FIRST ...
            throw new IllegalArgumentException("Name must not be empty.");
        }
        if (fuel < 0) {
            throw new IllegalArgumentException("Fuel must not be negative.");
        }
        this.name = name;                               // ... then assign
        this.flightType = flightType;
        this.fuel = fuel;
    }

    public String    getName()        { return name; }
    public FlightType getFlightType() { return flightType; }
    public double    getFuel()        { return fuel; }
}
```

Every "must not / cannot / is invalid" sentence in the task becomes a guard clause at the top of the constructor or method, throwing `IllegalArgumentException`. Guards come before any assignment, so a rejected object never half-exists.

Add `equals()` and `hashCode()` **only if** a test will compare two objects with `assertEquals`. Without them you compare references and every such test fails. If the task hands you an `equals()`, do not touch it.

Step 3 · The calculation that branches on the enum

This is the "if flight type equals this, divide the fuel by two" part. Three ways to write it — all correct, pick whichever the given code already uses.

```
// — A · switch expression (Java 14+). Cleanest, and the compiler checks you
//      covered every constant, so no default is needed.
public double calculateRequiredFuel(double distance) {
    return switch (flightType) {
        case SHORT_HAUL -> distance / 2;          // ← "divide by 2"
        case LONG_HAUL  -> distance * 2;
        case CARGO      -> distance * 3.5;
    };
}

// — B · classic switch. Note break, and default THROWS rather than returning 0.
public double calculateRequiredFuel(double distance) {
    switch (flightType) {
        case SHORT_HAUL: return distance / 2;
        case LONG_HAUL:  return distance * 2;
        case CARGO:      return distance * 3.5;
        default:         throw new IllegalStateException("Unknown flight type");
    }
}

// — C · if/else on the enum. Compare with == , never with .equals of a String.
public double calculateRequiredFuel(double distance) {
    if (flightType == FlightType.SHORT_HAUL) return distance / 2;
    if (flightType == FlightType.LONG_HAUL)  return distance * 2;
    return distance * 3.5;
}

// — D · if the enum carries the factor (Form B), there is NO branch at all.
//      This is the answer they are usually fishing for with "keep it extensible".
public double calculateRequiredFuel(double distance) {
    return distance * flightType.getFuelFactor();
}
}
```

Watch integer division. `distance / 2` is fine when `distance` is a `double`. If it is an `int`, `7 / 2` silently gives `3`. Write `distance / 2.0` whenever the result should be fractional.

Step 4 · The action that returns a boolean AND changes state

This is the method the "`assertTrue` + `assertEquals` in one test" instruction is pointing at. It answers two questions at once — *did it succeed?* and *what changed?*

```
public boolean launch(double distance) {
    double required = calculateRequiredFuel(distance);

    if (required > fuel) {
        return false;          // refuse – and change NOTHING
    }
    fuel -= required;          // state changes only on the success path
    return true;
}
```

The rule the graders test: on the failure path the object must be **completely unchanged**. Compute first, check, then mutate. Never mutate and roll back.

Step 5 · The tests — both assertions, one method

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class SpaceshipTest {

    // == THE SHAPE THEY ASKED FOR: assertTrue = "did it succeed",
    //    assertEquals = "and here is the state that proves it".
    @Test
    void testLaunchSuccessful() {
        // ARRANGE
        Spaceship ship = new Spaceship("Falcon", FlightType.SHORT_HAUL, 100.0);

        // ACT — one call, result captured
        boolean launched = ship.launch(40.0);    // needs 40 / 2 = 20

        // ASSERT — the return value ...
        assertTrue(launched);
        // ... and the state change it promised
        assertEquals(80.0, ship.getFuel(), 0.001); // 100 - 20, delta on doubles
    }

    @Test
    void testLaunchFailsWithoutEnoughFuel() {
        Spaceship ship = new Spaceship("Falcon", FlightType.LONG_HAUL, 10.0);

        boolean launched = ship.launch(40.0);    // needs 40 * 2 = 80 > 10

        assertFalse(launched);                  // refused ...
        assertEquals(10.0, ship.getFuel(), 0.001); // ... and nothing was consumed
    }

    // == the calculation itself, one case per enum constant
    @Test
    void testCalculateRequiredFuelPerFlightType() {
        assertEquals(50.0, new Spaceship("A", FlightType.SHORT_HAUL, 0).calculateRequiredFuel(100, 0.001);
        assertEquals(200.0, new Spaceship("B", FlightType.LONG_HAUL, 0).calculateRequiredFuel(100, 0.001);
        assertEquals(350.0, new Spaceship("C", FlightType.CARGO, 0).calculateRequiredFuel(100, 0.001);
    }

    // == the getters / initial state
    @Test
    void testConstructorStoresValues() {
        Spaceship ship = new Spaceship("Falcon", FlightType.CARGO, 500.0);
        assertEquals("Falcon", ship.getName());
        assertEquals(FlightType.CARGO, ship.getFlightType()); // enums compare fine
        assertEquals(500.0, ship.getFuel(), 0.001);
    }

    // == the guard clause
    @Test
    void testNegativeFuelThrowsException() {
        IllegalArgumentException ex = assertThrows(IllegalArgumentException.class,
            () -> new Spaceship("Falcon", FlightType.CARGO, -1.0));
        assertEquals("Fuel must not be negative.", ex.getMessage());
    }
}
```

Every assertion, and exactly when to reach for it

<code>assertEquals(expected, actual)</code>	values are equal. Expected first. Uses <code>.equals()</code> for objects, so the class needs <code>equals()</code> / <code>hashCode()</code>
<code>assertEquals(exp, act, 0.001)</code>	the only correct form for <code>double</code> / <code>float</code> — floating point rounding makes exact comparison unreliable
<code>assertTrue(condition)</code>	a boolean is true — the "was it successful" half
<code>assertFalse(condition)</code>	a boolean is false — the refusal case
<code>assertNull(x)</code> / <code>assertNotNull(x)</code>	reference is / is not null
<code>assertThrows(Ex.class, () -> ...)</code>	the lambda throws that type. The call must be inside the lambda. Returns the exception so you can check its message
<code>assertArrayEquals(exp, act)</code>	arrays, element by element
<code>assertSame(a, b)</code>	the same object reference, not merely equal
<code>assertAll(...)</code>	run several assertions and report every failure, not just the first
<code>@BeforeEach</code>	runs before every test — put shared setup here. JUnit builds a fresh test-class instance per test, so fields reset automatically

The checklist for this exercise

- 1 **Enum first**, constants spelled exactly as the task writes them, semicolon after the last one if it has a constructor.
- 2 **Fields** — `private`, `final` for anything that never changes.
- 3 **Constructor** — every "must not" becomes a guard clause *before* any assignment.
- 4 **Getters** — `public`, no setters unless the task asks for them.
- 5 **Calculation** — branch on the enum with `==` or `switch`; watch integer division.
- 6 **Boolean action** — compute, check, refuse-without-changing, or mutate and return `true`.
- 7 **Tests** — one per promise. `assertTrue` for the outcome plus `assertEquals` for the state, in the same method.
- 8 **Names** — copy required class and method names character-for-character from the task.

§2 · REST — save, find, remove

Three endpoints, one shape each. Everything on a plain background is fixed boilerplate; only the highlighted **{placeholders}** change.

The complete resource

```
@RestController
@RequestMapping("/api/{contacts}") // ← CHECK: does the given skeleton
public class {Contact}Resource { // already carry a path? If it only
    // sets consumes/produces, then EVERY
    private final {Contact}Service {contactService}; // method must spell out the full path.

    public {Contact}Resource({Contact}Service s) { this.{contactService} = s; }

    // = SAVE = POST · body carries the object · server assigns the id
    @PostMapping
    public ResponseEntity<{Contact}> save({Contact} @RequestBody {Contact} {contact}) {
        if ({contact}.getId() != null) {
            return ResponseEntity.badRequest().build(); // 400 – id is the server's job
        }
        return ResponseEntity.ok({contactService}.save({contact}));
    }

    // = FIND ONE = GET · id in the PATH · 404 when absent
    @GetMapping("/{ {contactId} }")
    public ResponseEntity<{Contact}> find({Contact} @PathVariable {Long} {contactId}) {
        return {contactService}.findById({contactId})
            .map(ResponseEntity::ok) // present → 200 + body
            .orElseGet(() -> ResponseEntity.notFound().build()); // absent → 404
    }

    // = FIND ALL / SEARCH = GET on the collection · filter is a QUERY parameter
    @GetMapping
    public ResponseEntity<List<{Contact}>> find({Contacts}) (
        @RequestParam(required = false) String {name}) {
        return ResponseEntity.ok({contactService}.find({name}));
    }

    // = REMOVE = DELETE · id in the PATH · empty body
    @DeleteMapping("/{ {contactId} }")
    public ResponseEntity<Void> remove({Contact} @PathVariable {Long} {contactId}) {
        if ({contactService}.findById({contactId}).isEmpty()) {
            return ResponseEntity.notFound().build(); // only if the task asks
        }
        {contactService}.remove({contactId});
        return ResponseEntity.noContent().build(); // 204 · or .ok().build() for 200
    }
}
```

THE SERVICE — PLAIN, AND ALMOST ALWAYS FIXED

```

@Service
public class {Contact}Service {

    private final Map<{Long}, {Contact}> store = new HashMap<>();
    private long nextId = 1;

    public {Contact} save({Contact} c) {
        if (c.getId() == null) { c.setId(nextId++); } // CREATE → new id
        store.put(c.getId(), c); // UPDATE → overwrite
        return c;
    }

    public Optional<{Contact}> findById({Long} id) { return Optional.ofNullable(store.get(id)); }

    public void remove({Long} id) { store.remove(id); }

    public List<{Contact}> find(String {name}) {
        if ({name} == null || {name}.isBlank()) {
            return new ArrayList<>(store.values()); // no filter → everything
        }
        return store.values().stream()
            .filter(c -> c.getName().equalsIgnoreCase({name}))
            .toList();
    }
}

```

The six decisions that decide your marks

QUESTION	ANSWER
Does <code>@RequestMapping</code> on the class already carry a path?	If it sets only <code>consumes</code> / <code>produces</code> , each method must spell out the full path (<code>@PostMapping("/contacts")</code>). Getting this wrong makes every test fail at once.
Path variable or request parameter?	Identity → path: <code>/contacts/{id}</code> . Filter, sort, page → query: <code>?name=Ann</code> .
Which verb?	save/create → <code>POST</code> · find → <code>GET</code> · full replace → <code>PUT</code> · remove → <code>DELETE</code> . The verb never appears in the URL.
Which status on delete?	<code>noContent()</code> = 204 if the task says "No Content", otherwise <code>ok()</code> = 200. Read the sentence.
Reject or throw?	<code>return ResponseEntity.badRequest().build()</code> or <code>throw new ResponseStatusException(HttpStatus.BAD_REQUEST)</code> . Both are accepted — match whichever the skeleton already imports.
Where does the logic go?	Resource = HTTP and validation only. Service = business rules. Repository = data access. Never reach from the resource into the repository.

Annotation reference

<code>@RestController</code>	on the class — every method returns the response body, not a view name
<code>@RequestMapping("/api/x")</code>	on the class — the shared path prefix for all its methods
<code>@GetMapping</code> <code>@PostMapping</code> <code>@PutMapping</code> <code>@DeleteMapping</code>	on the method — verb plus optional path suffix
<code>@PathVariable Long id</code>	binds <code>{id}</code> from the URL. The names must match exactly
<code>@RequestParam(required=false, defaultValue="X")</code>	binds <code>?param=</code> . Use for optional filters and sorting

<code>@RequestBody Contact c</code>	deserialises the JSON body into the object. POST and PUT only
<code>@Service</code> / <code>@Repository</code> / <code>@Component</code>	marks the class so Spring can inject it into the constructor
<code>ResponseEntity.ok(body)</code>	200 with a body
<code>ResponseEntity.noContent().build()</code>	204, empty
<code>ResponseEntity.notFound().build()</code>	404
<code>ResponseEntity.badRequest().build()</code>	400

If a client half is also asked for

```

webClient.post().uri("/contacts").bodyValue(contact)
    .retrieve().bodyToMono(Contact.class).block();           // SAVE, one object

webClient.get().uri("/contacts")
    .retrieve().bodyToFlux(Contact.class).collectList().block(); // FIND ALL, many

webClient.delete().uri("/contacts/{id}", id)
    .retrieve().bodyToMono(Void.class).block();             // REMOVE, no body

```

Nothing is sent until `.block()` or `.subscribe(...)` runs. A chain without one of those makes no request, raises no error, and the test fails on an empty result. `bodyToMono` = 0..1 object · `bodyToFlux` = 0..n, then `.collectList()`.

§3 · Deployment diagram

The question it answers: **which machine runs which piece of software, and over what protocol do the machines talk?** Nothing about time, nothing about classes.

Build order — six steps, in this sequence

- 1 **Every "X is a machine / device node" sentence → one 3-D box.** The 3-D shape is mandatory; a flat rectangle at the top level is marked wrong. Use «device» for physical hardware (a phone, a smart light, a sensor) and «machine node» or «executionEnvironment» for servers, VMs, JVMs, containers.
- 2 **Every "it contains / hosts / has deployed on it" → put those boxes INSIDE that node.** Nesting is the deployment relationship. Do not also draw an arrow from the artifact to its own node — that is a marked error.
- 3 **Every "communicates over <PROTOCOL>" → a plain solid line between the two NODES**, labelled with the protocol as a stereotype: «HTTPS», «TLS», «MQTT», «JDBC», «Wi-Fi». **No arrowhead. Ever.**
- 4 **Read the Hint block.** These tasks almost always end with a hint naming exactly which component requires which interface. Draw lollipop ○— and socket —○ between the **components**, never between the nodes.
- 5 **Name every interface** with the exact words from the problem statement — `User Accounts`, `Device Registry`. An interface captioned "Interface" scores nothing.
- 6 **Spell-check the stereotypes.** «Databass Server» and «Billling Client» are real deductions from a real paper.

The notation, drawn

Deployment diagram — what physically runs where



NODE (3-D box) = a physical or virtual machine. «device» = hardware · «executionEnvironment» = a container/VM/JVM running on it. ARTIFACT (plain box) = a deployable file — a .jar, .war, .sql, a Docker image. It is DEPLOYED ON a node, drawn nested inside it. The plain line between two nodes is a COMMUNICATION PATH — no arrowhead. Label it with the protocol as a «stereotype».

ELEMENT	DRAWN AS	MEANS
Node	3-D box	A computational resource — a machine, real or virtual.
«device»	stereotype on a node	Physical hardware: PC, phone, router, smart light, sensor.
«executionEnvironment»	a node <i>nested inside</i> a device	Software that hosts other software: JVM, Docker container, app server, OS.
Artifact	plain rectangle with «artifact», drawn inside a node	A concrete deployable file — .jar, .war, .sql, a container image. Nesting = "is deployed on".
Component	rectangle with «component», inside a node	A software unit running on that machine.
Communication path	plain line, node ↔ node, no arrowhead	A network connection. Labelled with the protocol .

ELEMENT	DRAWN AS	MEANS
Interface	lollipop ○— provided + socket —◁ required, component ↔ component	A service one component offers another. Labelled with the service name .
«deploy»	dashed arrow, artifact → node	The alternative to nesting when the layout makes nesting awkward. Either is accepted.

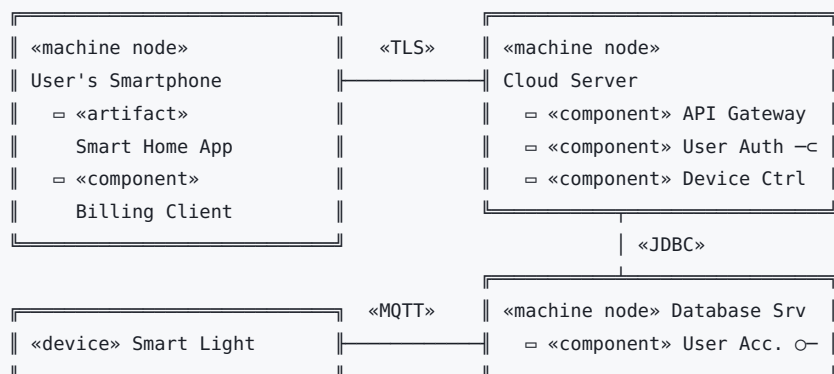
The distinction that is always tested

	COMMUNICATION PATH	INTERFACE
Connects	Node ↔ Node	Component ↔ Component
Looks like	plain solid line, no arrowhead	lollipop ○— cupped by socket —◁
Labelled with	the protocol — HTTPS, TLS, MQTT, JDBC, Wi-Fi	the service name — User Accounts, Device Registry

Drawing TLS as a lollipop is a real, marked mistake. TLS is how two *machines* talk, so it is a labelled line between two nodes. "User Accounts" is what one *component* offers another, so it is a lollipop and socket.

Worked example — read the sentence, draw the element

Statement	→ What you draw
"The user's smartphone is a machine node"	→ 3-D box «machine node» User's Smartphone
"It has the Smart Home App deployed on it"	→ □ «artifact» Smart Home App drawn INSIDE that box
"and contains the Billing Client component"	→ □ «component» Billing Client also inside
"The smartphone communicates with the cloud server over TLS"	→ == plain line between the two 3-D boxes, labelled «TLS»
"The Cloud Server hosts an API Gateway, a User Authentication Service and a Device Control Service"	→ three □ «component» boxes inside the Cloud Server node
"Hint: the User Authentication Service requires the User Accounts interface, provided by the database server"	→ —◁ socket on User Auth Service, ○— lollipop on the DB component, clipped together, labelled "User Accounts"
"The smart light is a device that talks to the hub over MQTT"	→ 3-D box «device» Smart Light, plain line labelled «MQTT»



== / — between NODES = communication path, labelled with the PROTOCOL
 ○— / —◁ between COMPONENTS = interface, labelled with the SERVICE name

Free marks people leave behind: the 3-D shape on every node · a protocol label on every line · a name on every interface. Each one is an independent scoring criterion.

§4 · Black-box testing

Three parts, always in this order: equivalence classes → one representative test per class → three-point boundaries. Marks are awarded per part, so **write all three even if one is rushed**.

Part 1 · Equivalence classes

An **equivalence class** is a set of inputs the system should treat identically — one test from the class stands for all of them. Give each parameter its own list, and label each class **Valid** or **Invalid**.

RULE	WHY IT EARNS MARKS
Split per bracket, per parameter combination.	If <i>standard</i> accounts are valid at 20–50 and <i>vip</i> at 10–80, you need classes for below / inside / above the bracket for each type . A single class whose verdict depends on the other parameter is a deduction.
"Valid" means accepted and evaluated — not that the answer is yes.	A level of 10 for a standard account is a perfectly <i>valid</i> input that produces <i>rejected</i> . Do not mark it Invalid.
Give every class an id.	ATC1 , CLC3 ... Part 2 refers back to them, and the coverage note needs them.
One catch-all Invalid class per parameter.	"any other value or empty input" — covers wrong strings, empty, null.

Part 1 – Equivalence Classes

accountType

```
ATC1 | Valid   | "standard"
ATC2 | Valid   | "vip"
ATC3 | Invalid  | any other value or empty input (e.g. "admin", "VIP", "", null)
```

characterLevel ← split per bracket; this is how you get to eight classes

```
CLC1 | Invalid | level <= 0          e.g. 0, -5
CLC2 | Valid   | standard, level 1..19 below the standard bracket
CLC3 | Valid   | standard, level 20..50 inside the standard bracket
CLC4 | Valid   | standard, level 51..100 above the standard bracket
CLC5 | Valid   | vip, level 1..9      below the vip bracket
CLC6 | Valid   | vip, level 10..80    inside the vip bracket
CLC7 | Valid   | vip, level 81..100   above the vip bracket
CLC8 | Invalid  | level >= 101        e.g. 101, 150
```

Part 2 · One representative test case per class

Part 2 – Representative Test Cases

```
TC1 | standard | 10 | rejected   TC6 | vip      | 90 | rejected
TC2 | standard | 35 | accepted   TC7 | standard | 0  | invalid
TC3 | standard | 75 | rejected   TC8 | vip      | 101 | invalid
TC4 | vip      | 5  | rejected   TC9 | admin    | 35 | invalid
TC5 | vip      | 45 | accepted
```

Coverage note – WRITE THIS T00, it is a separate scoring criterion:

```
TC1–TC3 cover ATC1 + CLC2/3/4
TC4–TC6 cover ATC2 + CLC5/6/7
TC7 covers CLC1 · TC8 covers CLC8 · TC9 covers ATC3
All eleven classes, nine cases.
```

Count before you write. If the task says "exactly N test cases", N is derivable: one per class, minus the overlaps. Two account types × three level brackets + two shared invalid level classes + one invalid type = 9. If your count differs, a class is missing or duplicated.

Part 3 • Boundary values — the three-point rule

For every limit, test **limit – 1 • limit • limit + 1**. Do it for **every** parameter combination the specification distinguishes, not just one.

Part 3 – Boundary Value Test Cases

VIP lower limit = 10, accountType = vip

TC10		9		rejected	← limit – 1
TC11		10		accepted	← ON the limit ★
TC12		11		accepted	← limit + 1

Standard upper limit = 50, accountType = standard

TC13		49		accepted	
TC14		50		accepted	← ON the limit ★
TC15		51		rejected	

TRAP	FIX
Using –1 / 0 / 1 around a lower limit	The three points go around the minimum valid value . If 1 is the smallest legal value, test 0 / 1 / 2 .
Doing the boundaries for only one user type	Do them for both . Each bracket has its own limits and each is scored.
Skipping the value exactly on the limit	★ The on-the-limit case is the only one that fails when <code>>=</code> is mistyped as <code>></code> . It is where the marks are.

Answer format that scores full marks: a plain aligned table, one row per case, columns `id` | `inputs` | `expected result`. No prose needed.

§5 · Fill in the blanks

Single words or short expressions. Spelling and capitalisation count, so learn the exact token.

JUnit

marks a test method	<code>@Test</code>
runs before / after every test	<code>@BeforeEach</code> · <code>@AfterEach</code>
runs once for the whole class — must be static	<code>@BeforeAll</code> · <code>@AfterAll</code>
skip a test	<code>@Disabled</code>
compare two values, expected first	<code>assertEquals(expected, actual)</code>
compare doubles	<code>assertEquals(expected, actual, delta)</code>
assert an exception is thrown	<code>assertThrows(Ex.class, () -> ...)</code>
the four test phases	Setup → Execute → Verify → Teardown
the three-part test structure	Arrange · Act · Assert

EasyMock

class-level annotation	<code>@ExtendWith(EasyMockExtension.class)</code>
the object under test	<code>@TestSubject</code> — must be instantiated inline
the collaborator	<code>@Mock</code> — leave unassigned; the extension injects it
record a call that returns	<code>expect(m.f(a)).andReturn(v);</code>
record a void call	<code>m.f(a); expectLastCall();</code>
record a throw	<code>expect(m.f()).andThrow(new Ex());</code>
switch from recording to playback	<code>replay(mock);</code>
assert every expectation happened	<code>verify(mock);</code>
the four phases	record → replay → exercise → verify
prove a call must not happen	do not record it, and keep the mock at the default type

Testing concepts

every line executed at least once	Statement coverage
every decision both true and false	Branch coverage
every atomic condition inside a compound, both ways	Term / condition coverage
every executable route through the method	Path coverage
decision points + 1	Cyclomatic complexity
human action → in the code → wrong internal state → observable	Mistake → Fault/Defect → Error → Failure
"did we build the product right?" (against the spec)	Verification

"did we build the right product?" (against the need)	Validation
never called, only fills a parameter	Dummy
returns canned values, verifies nothing	Stub
records calls so the test can inspect them later	Spy
expectations set in advance, verified afterwards	Mock
real but simplified implementation	Fake
one method in isolation	Unit test
several components working together	Integration test
whole system in a production-like environment, functional and non-functional	System test

Git, Gradle, CI

the three areas	Working Directory → Staging Area (Index) → Repository
edited but not <code>add</code> -ed	Modified · after <code>add</code> → Staged · after <code>commit</code> → Committed
create a branch and switch in one step	<code>git checkout -b name</code>
download without merging	<code>git fetch</code> · <code>git pull</code> = fetch + merge
publish a branch and set its upstream	<code>git push -u origin name</code>
resolve a conflict	edit the file, delete the markers, <code>git add</code> , <code>git commit</code>
compile, test and check in one command	<code>gradlew build</code> (= <code>assemble</code> + <code>check</code>)
fail-fast in a shell script	<code>set -e</code>
CI pipeline order	Build → Static Analysis → Unit → Integration → System → Acceptance
manual gate to production vs automatic	Continuous Delivery vs Continuous Deployment

UML & design

hollow triangle, solid line	Generalisation — points at the parent
hollow triangle, dashed line	Realisation — points at the interface
dashed line, open arrow	Dependency
hollow diamond	Aggregation — diamond sits at the whole , part survives
filled diamond	Composition — diamond at the whole , part dies with it
<code>-</code> <code>+</code> <code>#</code> <code>~</code>	private · public · protected · package-private
always runs · runs sometimes	<code>«include»</code> (arrow → included) · <code>«extend»</code> (arrow → base)
one flow becomes several concurrent flows	Fork · rejoined by a Join (which waits)
branches rejoin without waiting	Merge node
guards are written	in <code>[square brackets]</code> on the edge

open for extension, closed for modification	Open-Closed Principle
swap the algorithm at run time	Strategy
make an incompatible existing class fit	Adapter
notify a list of subscribers on change	Observer
skeleton in the superclass, steps in subclasses	Template Method

Last page · the nine lines

- 1 **Commit and push before you write anything**, then every ten minutes. Uncommitted work scores zero.
- 2 **Unit testing**: enum → fields → constructor with guards → getters → calculation → boolean action → tests. In that order.
- 3 **One test method can hold both** `assertTrue(result)` for "did it succeed" and `assertEquals(expected, state, delta)` for "what changed".
- 4 **Every double comparison needs a delta**. Every `assertEquals` has expected first.
- 5 **The throwing call goes inside the `assertThrows` lambda**, never before it.
- 6 **REST**: check whether `@RequestMapping` already carries a path. Identity in the path, filters in query parameters. POST rejects a supplied id.
- 7 **Deployment**: 3-D boxes for nodes, nesting means deployed-on, plain labelled lines between nodes, lollipop and socket only between components.
- 8 **Black-box**: classes split per bracket **per parameter combination**; boundaries are `limit-1` / `limit` / `limit+1` around the minimum **valid** value, for every type.
- 9 **Copy required names character-for-character** out of the task before writing any body.